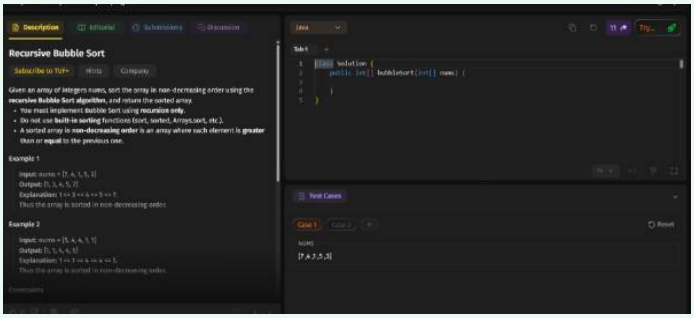
ChatGPT



Teach me Recursive Bubble Sort

Compare with other AI

Absolutely. Let's learn **Recursive Bubble Sort** from the ground up, using the exact Java problem in your screenshot.

1. First: normal Bubble Sort

Suppose we have:

`[7, 4, 1, 5, 3]`

Bubble Sort repeatedly compares **neighboring elements**:

- Compare `7` and `4` → swap → `[4, 7, 1, 5, 3]`
- Compare `7` and `1` → swap → `[4, 1, 7, 5, 3]`
- Compare `7` and `5` → swap → `[4, 1, 5, 7, 3]`
- Compare `7` and `3` → swap → `[4, 1, 5, 3, 7]`

Notice something important:

> After one complete pass, the **largest element reaches the end**.

So after the first pass:

`[4, 1, 5, 3, 7]`

We don't need to touch `7` anymore.

---

2. Where does recursion come in?

Instead of writing a loop for every pass, we make a recursive function.

Think:

```
bubbleSort(array)
    ↓
```

```
put largest element at the end
    ↓
bubbleSort(smaller part)
    ↓
put next largest at the end
    ↓
bubbleSort(smaller part)
    ↓
...
```

So recursion replaces the **outer loop**.

The key idea is:

```
n elements
→ perform one Bubble Sort pass
→ largest element is now at index n-1
→ recursively sort first n-1 elements
```

3. The recursive structure

In Java, we can write:

```Java
public int[] bubbleSort(int[] nums) {
    bubble(nums, nums.length);
    return nums;
}

private void bubble(int[] nums, int n) {
    // base case
    if (n == 1) {
        return;
    }

    // one bubble-sort pass
    for (int i = 0; i < n - 1; i++) {
        if (nums[i] > nums[i + 1]) {
            int temp = nums[i];
            nums[i] = nums[i + 1];
            nums[i + 1] = temp;
        }
    }

    // recursively sort remaining elements
    bubble(nums, n - 1);
}
```

Let's understand **every part**.

---

4. The base case

```Java
if (n == 1) {
    return;
}
```

Recursion needs a condition telling it when to stop.

If only one element remains, it's already sorted.

For example:

```
[1]
```

There's nothing to compare, so we're done.

---

5. The Bubble Sort pass

This part:

```java
for (int i = 0; i < n - 1; i++) {
    if (nums[i] > nums[i + 1]) {
        int temp = nums[i];
        nums[i] = nums[i + 1];
        nums[i + 1] = temp;
    }
}
```

is the **actual Bubble Sort operation**.

For:

```
[7, 4, 1, 5, 3]
```

we compare:

```
7 > 4 → swap
4 > 1 → swap
7 > 5 → swap
7 > 3 → swap
```

Result:

```
[4, 1, 5, 3, 7]
```

Now `7` is guaranteed to be in its correct position.

---

6. The recursive call

Then we do:

```java
bubble(nums, n - 1);
```

Why `n - 1`?

Because the last element is already sorted.

We started with:

```
[4, 1, 5, 3, 7]
                ↑
```

```
              finished
```

So we only need to work on:

```
[4, 1, 5, 3]
```

The next recursive call performs another pass:

```
[4, 1, 5, 3]
```

After the pass:

```
[1, 4, 3, 5, 7]
         ↑  ↑
         5 is now correct
```

Then:

```
bubble(nums, 3)
```

Another pass:

```
[1, 3, 4, 5, 7]
```

Then:

```
bubble(nums, 2)
```

Already sorted.

Then:

```
bubble(nums, 1)
```

Base case → stop.

---

7. The most important mental model

Remember this:

```
bubble(nums, n)

1. Sort ONE pass
2. Largest element goes to position n-1
3. Recursively solve n-1 elements
```

That's recursive Bubble Sort.

You can visualize it like this:

```
bubble([7,4,1,5,3], 5)
        ↓
     [4,1,5,3,7]
               ↑ fixed
```

```
bubble([4,1,5,3,7], 4)
        ↓
     [1,4,3,5,7]
              ↑ fixed

bubble([1,4,3,5,7], 3)
        ↓
     [1,3,4,5,7]
            ↑ fixed

bubble([1,3,4,5,7], 2)
        ↓
     [1,3,4,5,7]

bubble(..., 1)
        ↓
      STOP
```

8. Why can't we just make everything recursive?

Your problem says:

> "Implement Bubble Sort using recursion only."

Usually, the intended solution is to make the **passes recursive**, while using a loop for the neighboring comparisons.

So this is a common and clean solution:

```Java
private void bubble(int[] nums, int n) {
    if (n == 1) return;

    for (int i = 0; i < n - 1; i++) {
        if (nums[i] > nums[i + 1]) {
            int temp = nums[i];
            nums[i] = nums[i + 1];
            nums[i + 1] = temp;
        }
    }

    bubble(nums, n - 1);
}
```

The **outer repetition** is recursion.

---

9. One optimization you should know

We can track whether any swap happened:

```Java
private void bubble(int[] nums, int n) {
    if (n == 1) {
        return;
    }

    boolean swapped = false;

    for (int i = 0; i < n - 1; i++) {
        if (nums[i] > nums[i + 1]) {
```

```
            int temp = nums[i];
            nums[i] = nums[i + 1];
            nums[i + 1] = temp;
            swapped = true;
        }
    }

    if (!swapped) {
        return;
    }

    bubble(nums, n - 1);
}
```

If we have:

```
[1, 2, 3, 4, 5]
```

the first pass makes **zero swaps**.

That means the array is already sorted, so we can immediately stop.

---

10. What you should remember for interviews/exams

The entire algorithm boils down to three things:

Base case

```
Java
if (n == 1) return;
```

Bubble one pass

```
Java
for (int i = 0; i < n - 1; i++) {
    if (nums[i] > nums[i + 1]) {
        // swap
    }
}
```

Recursion

```
Java
bubble(nums, n - 1);
```

And the intuition is:

> **One pass puts the largest element at the end. Then recursively sort everything before it.**

For your screenshot, the clean submission would be:

```
Java
class Solution {
    public int[] bubbleSort(int[] nums) {
        bubble(nums, nums.length);
        return nums;
```

```
    }

    private void bubble(int[] nums, int n) {
        if (n <= 1) {
            return;
        }

        for (int i = 0; i < n - 1; i++) {
            if (nums[i] > nums[i + 1]) {
                int temp = nums[i];
                nums[i] = nums[i + 1];
                nums[i + 1] = temp;
            }
        }

        bubble(nums, n - 1);
    }
}
```

Time: $O(n^2)$ worst/average case

Extra space: $O(n)$ because of the recursion stack.

The trick to remember is **"pass → largest goes home → reduce n → recurse."**